

Longest Common Subsequence (LCS)

- * Using Recursion
- * Using Memorization
- * Using Dynamic Problem

S_1 : a b c d e f g h i j

S_2 : i d g i

Not to check whether the S_2 is in S_1 or not

But to which set of characters are coming in seq.

c d g i is the ^{Longest} seq present in S_1

d g i

g i

S_1 a b c d e f g h i j

S_2 e c d g i

should not intersect

e g i

c d g i — Longest subseq

S_1 : a b d a c e

S_2 : b a b c e

① b a c e

② a b c e

Alg

```
int LCS(i, j)
{
    if A[i] == '\0' || B[j] == '\0'
        return 0;
```

```
    else if (A[i] == B[j])
        return (1 + LCS(i+1, j+1));
```

```
    else
        return max(LCS(i+1, j), LCS(i, j+1));
```

}

A [b | d | \0]
0 1 2

B [a | b | c | d | \0]
0 1 2 3 4

→ Eg 2

Str 1: stone — m

Str 2: longest — n

		0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0	0
S 1	0	0	0	0	0	0	0	1	1
T 2	0	0	0	0	0	0	0	1	2
O 3	0	0	1	1	1	1	1	1	2
N 4	0	0	1	2	2	2	2	2	2
E 5	0	0	1	2	2	3	3	3	3
		O	N			E			

Longest Common Subseq size = 3

ONE is the Longest Common Subseq.

$$O(m \times n) = O(mn)$$